

Connect with us

We're waiting to hear from you. Connect with us on our Facebook page (facebook.com/devworx.in) and Twitter (@devworx). We also have a group on LinkedIn (linkd.in/devworx). Network with fellow developers!

A Google a day

Sharpen your googling skills with one Google search a day
<http://goo.gl/jUH5>

Developer corner

```
models.ForeignKey(Person,
blank=True, null=True)
    borrowed_on = models.
DateField(blank=True, null=True)
    def __unicode__(self):
        return self.name
```

These lines describe the data structure of our web site. The Person class defines the person and the Book class defines the book. The first name and last name are character fields of specified length. The phone number is stored in a character field as well, in case you wish to store numbers as "+91 99 9999 9999" and the email is an email field, which is optimised for the purpose.

In the Book class, the name is a character field with a much larger limit on length. There's a `borrowed_on` field to store the date on which it was borrowed. Finally, you have `borrowed_by`, which relates the Book to its borrower. We can directly use Person here as it's already defined, but otherwise we would have to enter it as "Person". With this kind of relation, one Book can only go to one person, but one person could have borrowed multiple books.

By setting `blank=True` we've made entering those fields optional. So if you want, you can just enter the first name of a person, with an option to enter a last name, a phone and an email as well. Similarly for a book, all you need is a name. Not all books will be borrowed, and once a book is returned, it will need to have no one associated with it. In addition to `blank=True`, we need to set `null=True` for `ForeignKey` and `DateField` for this to work.

Finally, the `__unicode__` function describes the string representation of an object. If you were to remove that, every book would be called "Book object" in the admin panel.

7. Add the Book and Person objects to the admin panel

Create a file called `admin.py` in your books app directory, and add the following to it:

```
from books.models
import Book, Person
from django.contrib import admin
admin.site.register(Book)
admin.site.
register(Person)
```

This code registers the Book and Person classes with the admin site, so they can be managed using the automatic admin backend.

8. Run the syncdb command

Now, in your project directory in the command line run the following command:

```
python manage.py syncdb
```

This command will create the database and the basic structure required by each application. It will also ask you to enter a username, email and password for the default site administrator. If you modify your application, you'll need to delete your database file and run this command again. You'll lose your data in your process though! There are other ways to accomplish this without deleting your data but they are beyond the scope of this tutorial.

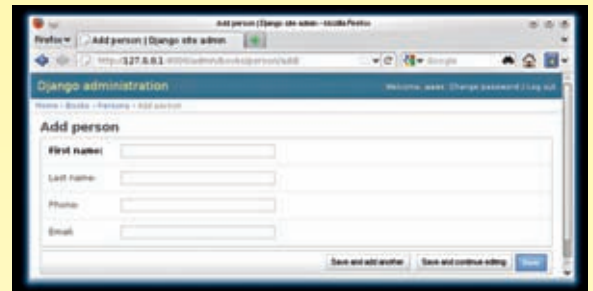
9. Start your server

In the command line in your project directory run the following command:

```
python manage.py runserver
```

This is not a production worthy

server, but is good enough for testing. By default, it will launch a local server that you can access by entering `127.0.0.1:8000` in the browser address bar. The server is smart enough to automatically restart if it detects that you have changed your code. It won't detect if you have added new files, and might stop working if you save a file with an error though.



Adding a Person

10. Use your web site

Enter `http://127.0.0.1:8000/admin` in your browser address bar, enter the username and password you picked earlier. You'll now be able to access a fully functional admin panel that will let you browse and edit the books and person you have added.

You're nearly done! Now all you have is a web site that has an admin panel, but not a frontend. In this example, it might be enough to simply view all details in the admin panel, but a normal web site will have a frontend. For that you need to learn how Django views and how templates work which we'll cover in a future article.

You can see how easily we have a web site up with customised functionality. You could now easily modify this site and add an owner for each book so it works as a book management system that works for multiple people. 



Ching's Secret

Instant, Hakka & Pad Thai Noodles,
Soups, Sauces & Miracle Masala

Ching's Khao, Baki Bhool Jao



@ChingsSecret



www.facebook.com/ChingsSecret



www.youtube.com/ChingsSecret